



LeetCode 767: Reorganize String

1. Problem Title & Link

Title: LeetCode 767: Reorganize String

Link: <https://leetcode.com/problems/reorganize-string/>

2. Problem Statement (Short Summary)

Given a string:

`s`

Rearrange its characters so that:

No two adjacent characters

are the same.

Return any valid rearranged string.

If it is impossible:

return ""

3. Examples (Input → Output)

Example 1

Input:

`s = "aab"`

Output:

`"aba"`

Explanation:

No adjacent characters

are equal.

Example 2

Input:

`s = "aaab"`

Output:

`""`

Explanation:

Impossible

because:

`a` appears 3 times

but only 1 non-`a` character exists.

Example 3



Input:

`s = "vvvlo"`

Output:

`"vlvov"`

One valid answer.

4. Constraints

- $1 \leq s.length \leq 500$
- `s` consists of lowercase English letters

Need a valid arrangement or determine impossibility.

5. Core Concept (Pattern / Topic)

Greedy + Max Heap

Related Topics:

- Priority Queue
- Frequency Counting
- Greedy Scheduling

6. Thought Process (Step-by-Step Explanation)

Brute Force

Generate all permutations.

Check:

No adjacent characters equal

Complexity:

$O(n!)$

Impossible.

Key Observation

Characters with highest frequency are the most dangerous.

Example:

`"aaabbc"`

If we don't place:

`'a'`

carefully,

we may end up with:

`aa`

adjacent.



Greedy Idea

Always choose the character with highest remaining frequency.

But:

Don't use the same character

as previous position.

This suggests:

Max Heap

Strategy

1. Count frequencies.
2. Put characters into a Max Heap.
3. Pick the most frequent character.
4. Hold the previous character temporarily.
5. Use another character next.
6. Reinsert previous character if frequency remains.

Why It Works

The character with highest frequency is always placed as early as possible.

This prevents large frequency characters from accumulating and creating adjacent duplicates later.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
s = "aaabbc"
Frequencies:
a → 3
b → 2
c → 1
Heap:
(a, 3)

(b, 2)

(c, 1)
Build:
a

ab

aba

abab
```

```
ababa  
  
ababac  
Valid.
```

8. Pseudocode (Language Independent)

```
Count frequencies  
  
Create max heap  
  
prev = null  
  
while heap not empty:  
    take highest frequency char  
    add to answer  
    decrease frequency  
    push previous back if needed  
    store current as previous  
  
If length(answer) != length(s):  
    return ""  
  
Return answer
```

9. Code Implementation

 Python

```
from collections import Counter  
import heapq  
  
class Solution:  
    def reorganizeString(self, s):  
        freq = Counter(s)  
  
        heap = [  
            (-count, char)  
            for char, count in freq.items()  
        ]
```



```
heapq.heapify(heap)

result = []

prev_count = 0
prev_char = ""

while heap:

    count, char = heapq.heappop(heap)

    result.append(char)

    count += 1

    if prev_count < 0:
        heapq.heappush(
            heap,
            (prev_count, prev_char)
        )

    prev_count = count
    prev_char = char

answer = "".join(result)

if len(answer) != len(s):
    return ""

return answer
```

✓ Java

```
import java.util.*;

class Solution {

    public String reorganizeString(String s) {

        Map<Character, Integer> freq =
            new HashMap<>();

        for (char ch : s.toCharArray()) {

            freq.put(
                ch,
                freq.getOrDefault(ch, 0) + 1
            );
        }

    }

}
```



```
PriorityQueue<int[]> pq =
    new PriorityQueue<>(
        (a, b) -> b[0] - a[0]
    );

for (char ch : freq.keySet()) {

    pq.offer(
        new int[]{
            freq.get(ch),
            ch
        }
    );
}

StringBuilder result =
    new StringBuilder();

int prevFreq = 0;
char prevChar = '#';

while (!pq.isEmpty()) {

    int[] current = pq.poll();

    int count = current[0];
    char ch = (char) current[1];

    result.append(ch);

    count--;

    if (prevFreq > 0) {

        pq.offer(
            new int[]{
                prevFreq,
                prevChar
            }
        );
    }

    prevFreq = count;
    prevChar = ch;
}

if (result.length() != s.length())
    return "";

return result.toString();
```



```
}  
}
```

10. Time & Space Complexity

Time Complexity

Let:

n = length of string

Each character enters and leaves heap.

$O(n \log 26)$

Since:

26 letters only

Simplifies to:

$O(n)$

Space Complexity

$O(26)$

or

$O(1)$

11. Common Mistakes / Edge Cases

Mistake 1

Not checking impossible cases.

Example:

"aaaaab"

Impossible.

Mistake 2

Always taking top character without holding previous.

Can create:

aa

adjacent.

Mistake 3

Using sorting repeatedly.

Results in unnecessary overhead.

12. Detailed Dry Run (Step-by-Step Table)



Input:

s = "aab"

Frequencies:

Character	Count
a	2
b	1

Heap:

(a,2)

(b,1)

Iteration 1

Take:

a

Result:

"a"

Remaining:

a=1

Hold:

(a,1)

Iteration 2

Take:

b

Result:

"ab"

Push back:

(a,1)

Iteration 3

Take:

a

Result:

"aba"

Done.



13. Common Use Cases (Real-Life / Interview)

Task Scheduling

Prevent same tasks from running consecutively.

CPU Scheduling

Separate identical processes.

Event Management

Avoid repeated event types.

Data Transmission

Prevent repeated signal patterns.

14. Common Traps (Important!)

- Forgetting previous character handling
- Not reinserting remaining frequency
- Ignoring impossible cases
- Using queue instead of max heap

15. Builds To (Related LeetCode Problems)

1. LC 621 — Task Scheduler
2. LC 358 — Rearrange String k Distance Apart
3. LC 767 — Reorganize String
4. LC 1405 — Longest Happy String
5. LC 1054 — Distant Barcodes

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Permutations	$O(n!)$	$O(n)$	Impossible
Sorting Repeatedly	$O(n^2 \log n)$	$O(n)$	Slow
Max Heap + Greedy	$O(n \log 26)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

The most frequent character is always the hardest to place. By always choosing the currently most frequent available character and temporarily holding the previously used one, we guarantee that no two adjacent characters become equal.



18. Variations / Follow-Up Questions

Follow-Up 1

What if characters must be at least:

k distance apart

Use:

LC 358 – Rearrange String k Distance Apart

with:

Max Heap + Queue

Follow-Up 2

What if uppercase letters are included?

No change in logic.

Follow-Up 3

What if we need lexicographically smallest valid answer?

Use:

Heap ordered by

frequency + character

with additional tie-breaking rules.